

DBMS and SQL – Complete Study Material

◆ 1. Introduction to DBMS

A **Database Management System (DBMS)** is a software that stores, organizes, and manages access to data. It acts as an interface between the database and the users or applications.

Real-World Example:

A university database managing students, courses, enrollments, etc. It ensures students don't register for the same course twice or get enrolled in a non-existent course.

Key Functions:

- **Storage:** Stores data in structured format
 - **Retrieval:** Query language to fetch data (SQL)
 - **Security:** Controls access to data
 - **Backup and Recovery:** Ensures durability
 - **Concurrency Control:** Manages simultaneous data access
-

◆ 2. Data Abstraction in DBMS

DBMS uses **data abstraction** to hide low-level implementation details.

Levels of Abstraction:

1. **Physical Level:** How data is physically stored (files, indexes)
 2. **Logical Level:** What data is stored (tables, relationships)
 3. **View Level:** What part of data is visible to users (customized views)
-

◆ 3. Data Models and Keys

Types of Keys:

- **Primary Key:** Uniquely identifies each record

 *Example:* EmployeeID in Employee table

- **Foreign Key:** References primary key in another table
 *Example:* DepartmentID in Employee referencing Department table
 - **Candidate Key:** Any field(s) that can uniquely identify a record
 - **Alternate Key:** Candidate key not chosen as Primary
 - **Composite Key:** Key formed from more than one column
-

◆ 4. SQL Overview

SQL (Structured Query Language) is used to manage relational databases.

◆ 5. SQL Sub-languages

1. DDL (Data Definition Language)

Used to define schema.

```
CREATE TABLE Employee (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    salary DECIMAL(10, 2)  
);
```

```
ALTER TABLE Employee ADD department_id INT;
```

```
DROP TABLE Employee;
```

2. DML (Data Manipulation Language)

Used for data handling.

```
INSERT INTO Employee (id, name, salary) VALUES (1, 'John', 50000);
```

```
UPDATE Employee SET salary = 55000 WHERE id = 1;
```

```
DELETE FROM Employee WHERE id = 1;
```

3. DQL (Data Query Language)

Used to query data.

```
SELECT * FROM Employee;
```

```
SELECT name FROM Employee WHERE salary > 40000;
```

4. DCL (Data Control Language)

Controls access.

```
GRANT SELECT, INSERT ON Employee TO user1;
```

```
REVOKE INSERT ON Employee FROM user1;
```

5. TCL (Transaction Control Language)

Handles transactions.

```
COMMIT;
```

```
ROLLBACK;
```

```
SAVEPOINT Save1;
```

◆ 6. Filtering and Sorting Data

-- Filtering

```
SELECT * FROM Employee WHERE department_id = 2;
```

-- Sorting

```
SELECT * FROM Product ORDER BY price ASC;
```

◆ 7. SQL Constraints

Ensure **data integrity**:

- NOT NULL – field must have value
- UNIQUE – all entries must be different
- PRIMARY KEY – unique + not null

- FOREIGN KEY – ensures valid references
- CHECK – validation rule
- DEFAULT – assigns default value

```
CREATE TABLE Student (
    id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT CHECK (age > 10),
    grade CHAR(1) DEFAULT 'A'
);
```

◆ 8. SQL Joins

Types of Joins:

- **INNER JOIN:** Only matching rows
- **LEFT JOIN:** All from left + matching from right
- **RIGHT JOIN:** All from right + matching from left
- **FULL OUTER JOIN:** All from both sides
- **NATURAL JOIN:** Auto join by same column names
- **CROSS JOIN:** Cartesian product

Example:

```
SELECT e.name, d.name
FROM Employee e
JOIN Department d ON e.department_id = d.id;
```

◆ 9. Aggregate Functions & Grouping

Common Functions:

- COUNT() – number of rows

- SUM() – total
- AVG() – average
- MAX() – max value
- MIN() – min value

 Example:

```
SELECT department_id, COUNT(*)  
FROM Employee  
GROUP BY department_id;
```

 Using HAVING:

```
SELECT department_id, AVG(salary)  
FROM Employee  
GROUP BY department_id  
HAVING AVG(salary) > 50000;
```

◆ **10. Subqueries**

A subquery is a query inside another query.

```
SELECT name  
FROM Employee  
WHERE salary = (SELECT MAX(salary) FROM Employee);
```

◆ **11. Window Functions**

Used for advanced analytics.

```
SELECT name, salary,  
       RANK() OVER (ORDER BY salary DESC) AS salary_rank  
FROM Employee;
```

Functions include:

- ROW_NUMBER()
 - RANK()
 - DENSE_RANK()
 - NTILE(n)
-

◆ 12. NULL Handling

-- Check for NULL

```
SELECT * FROM Orders WHERE delivery_date IS NULL;
```

◆ 13. Transactions

A transaction = unit of work with **ACID** properties:

- **Atomicity:** All or none
- **Consistency:** Valid state
- **Isolation:** No intermediate changes seen
- **Durability:** Persisted once committed

```
BEGIN;
```

```
UPDATE Employee SET salary = 60000 WHERE id = 1;
```

```
COMMIT;
```

-- Or

```
ROLLBACK;
```

◆ 14. Normalization

Eliminates redundancy and improves consistency.

Form	Description
1NF	Atomic columns (no repeating groups)
2NF	1NF + full dependency on primary key
3NF	2NF + no transitive dependency
BCNF	3NF + every determinant is candidate key

Real-World Analogy:

Breaking a wide spreadsheet into smaller, linked tables so each fact is stored only once.

◆ 15. Real-World SQL Examples

Find Second Highest Salary

```
SELECT MAX(salary)
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);
```

Average Salary by Department

```
SELECT department_id, AVG(salary)
FROM Employee
GROUP BY department_id;
```

Employees Without Department

```
SELECT e.name
FROM Employee e
LEFT JOIN Department d ON e.department_id = d.id
WHERE d.id IS NULL;
```

✓ Summary

Topic	Key Takeaways
DBMS	Interfaces with users, supports CRUD, security
SQL	Powerful for data query and modification
Joins	Combine relational data
Constraints	Enforce data quality
Normalization	Avoid redundancy
Transactions	Ensure data integrity
Functions	Simplify summarization